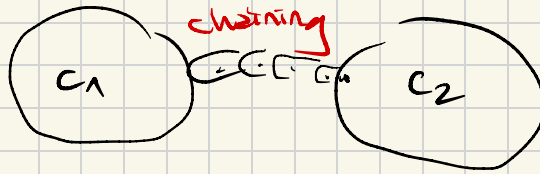


pairs



Downside: prone to "chaining", sensitive to noise

2) Complete linkage, maximum pairwise distance

$$d(C_k, C_{k'}) = \max_{i \in C_k, j \in C_{k'}} d(x_i, x_j)$$

→ computes "worst case" between pairwise clusters, then finds "smallest worst case"

3) Average linkage: mean of all pairwise distances

$$d(C_k, C_{k'}) = \frac{1}{|C_k| |C_{k'}|} \sum_{i \in C_k} \sum_{j \in C_{k'}} d(x_i, x_j)$$

4) Representative linkage: distance between cluster representatives

$$d(C_k, C_{k'}) = d(\text{repr}(C_k), \text{repr}(C_{k'}))$$

→ representative could be defined by mean,

median etc.

Properties and tradeoffs:

- 1) Single linkage: sensitive to outliers and chaining
- 2) Complete linkage: less sensitive to chaining, but may break "closeness" property (we want small within as compared to between cluster distances)
- 3) Average linkage: balanced behavior between 1 & 2
- 4) Representative linkage: easy to interpret, but depends on repr. choice

5.2.2 K-means Clustering

- Assuming K (number of clusters) is predefined
- Consider the total pairwise distance w/in the dataset:

$$d_{\text{total}} = \frac{1}{2} \sum_{i=1}^N \sum_{i'=1}^N d(x_i, x_{i'})$$

- Any clustering divides d_{total} into:

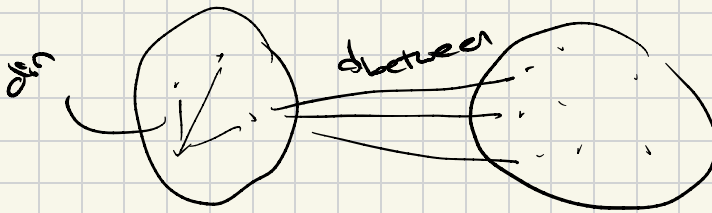
1) within cluster distance d_{in}

2) between cluster distance $d_{between}$

$$d_{total} = d_{in} + d_{between}$$

$$d_{in} = \frac{1}{2} \sum_{k=1}^K \sum_{i \in C_k} \sum_{i' \in C_k} d(x_i, x_{i'})$$

$$d_{between} = \frac{1}{2} \sum_{k=1}^K \sum_{i \in C_k} \sum_{i' \notin C_k} d(x_i, x_{i'})$$



- A good clustering algorithm minimizes d_{in} , or equivalently maximizes $d_{between}$
- In theory, we can find best clustering by exhaustive search, but computationally infeasible for large datasets
- K-means algorithm is a fast heuristic to approximately minimize d_{in} assuming

$$d(x_i, x_{i'}) = \|x_i - x_{i'}\|_2^2$$

- In this case the optimal cluster representative is the cluster mean

$$\bar{x}_k = \frac{1}{N_k} \sum_{i \in C_k} x_i$$

- Introduce hidden variables $y_i \in \{1, \dots, K\}$ indicating cluster assignment, so $y_i = k$ means $x_i \in C_k$.

Algorithm:

- 1) Initialize cluster centers $\bar{x}_k^{(0)}$ for $k=1 \dots K$
- 2) Repeat until convergence: for $t=1 \dots T/\text{conv.}$

(a) Assignment step: assign each point to closest cluster

$$y_i^{(t)} = \underset{k}{\operatorname{argmin}} \|x_i - \bar{x}_k^{(t-1)}\|_2^2$$

(b) Update step: recompute cluster means based on new assignments



$$\bar{x}_k^{(t)} = \frac{1}{N_k^{(t)}} \sum_{i: y_i^{(t)} = k} x_i$$

- will converge to local minimum in d_{tot} , but does not guarantee global minimum
- will strongly depend on initialization
- A common and effective strategy for initialization:

K-Medus ++

1) choose the first center $\bar{x}_1^{(0)}$ uniformly at random from dataset

2) for $k=2, \dots, K$, repeat:

- compute distance to the nearest current center for each point

$$d_i = \min_{j=1, \dots, k-1} \|x_i - \bar{x}_j^{(0)}\|_2^2$$

(obtain a vector $d = [d_1 \dots d_N]$)

- Normalize distances to form probabilities

$$p_i = \frac{d_i}{\sum_j d_j}$$

- Sample next center $\bar{x}_k^{(b)}$ from $\{x_i\}$ according to p_i

→ this will sample data points with large minimum distance to all other cluster means

5.2.3 K-medoids clustering

- variant of K-means with representative being the medoid, working with any type of distance d
- downside: computationally expensive
- advantage: more robust to outliers

5.2.4 How to choose K ?

- there is no universally optimal method, just several heuristics
- The most popular one is "elbow method":
 - 1) compute d_{in} for various values of K
 - 2) As K decreases, d_{in} will increase

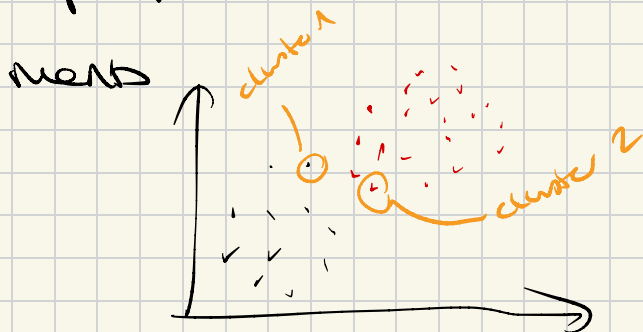
3) Plot d_W w K and look for point where curve "bends", called the elbow



- other methods include:
 - silhouette score
 - gap statistic
 - cross-validation approaches

5.2.4 Gaussian Mixture Models (GMMs)

- recap: k-means receives hard cluster assignments



- GMMs are generative models:

- 1) Yield soft assignment for each x_i :

$$P(\underbrace{y_i}_{\text{not observed "latent" label}} = k \mid x_i)$$

not observed "latent" label

- 2) Model posterior

$$p(y \mid x_i) = \frac{p(x_i \mid y_i) p(y_i)}{p(x_i)}$$

shape of clusters prior of x_i belonging to cluster k

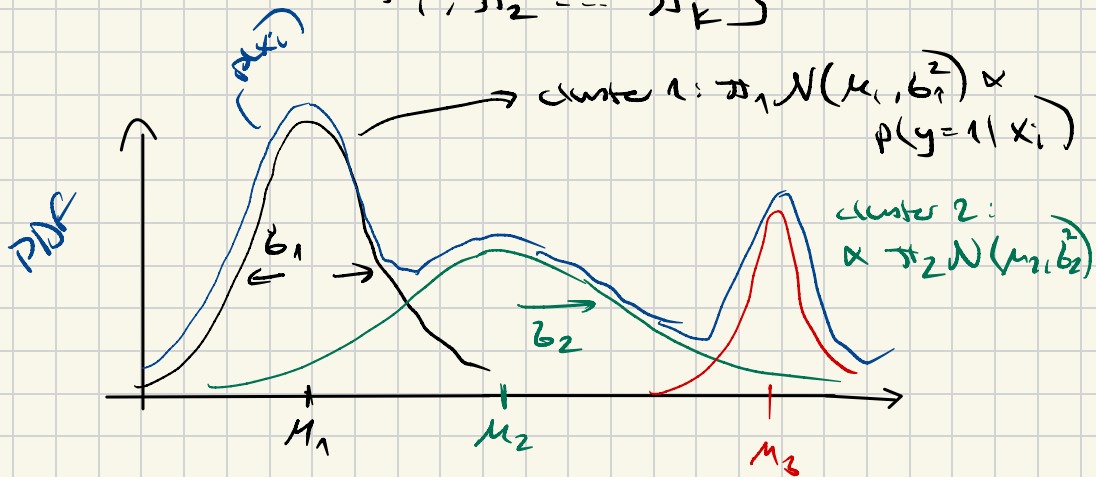
Choice for shape:

$$p(x_i \mid y_i = k) = \mathcal{N}(\mu_k, \Sigma_k)$$

$$p(y_i = k) = \pi_k \approx \frac{N_k}{N} \quad \sum_k \pi_k = 1$$

Total parameter set:

$$\Theta = \{\mu_1, \mu_2, \dots, \mu_K, \Sigma_1, \Sigma_2, \dots, \Sigma_K, \pi_1, \pi_2, \dots, \pi_K\}$$



$$p(x_i) = \sum_k \pi_k \mathcal{N}(\mu_k, \Sigma_k)$$

MAXIMUM LIKELIHOOD ESTIMATION
Optimization (loss function)

- set up likelihood and optimize Θ

$$\hat{\Theta} = \underset{\Theta}{\operatorname{argmax}} \prod_{i=1}^N p_{\Theta}(x_i) \quad \text{find all.}$$

$$= \underset{\Theta}{\operatorname{argmax}} \sum_{i=1}^N \log p_{\Theta}(x_i)$$

$$= \underset{\Theta}{\operatorname{argmax}} \sum_{i=1}^N \log \left(\sum_{k=1}^K \pi_k \mathcal{N}(\mu_k, \Sigma_k) \right)$$

- Problem: sum in the log-likelihood since we don't know true cluster assignment, to get a hold of this introduce a "latent variable" that will assign to a cluster:

"one-hot encoding" $\begin{cases} z_{i,k} = 1 \text{ if } x_i \text{ comes from } N(\mu_k, \Sigma_k) \\ z_{i,\text{else}} = 0 \end{cases}$

$$\Rightarrow p_{\theta}(x_i) = \sum_k^K \pi_k N(\mu_k, \Sigma_k) = \sum_{k=1}^K p(z_{i,k}=1 | \theta) \cdot p(x_i | z_{i,k}=1)$$

- Now: by rewriting the problem we introduced new random variables $z_i, i=1 \dots N$.
- $X := \{x_1, x_2, \dots, x_N\}$; $z := \{z_1, z_2, \dots, z_N\}$
- Now our ^{log} likelihood becomes:

$$\log p_{\theta}(X) = \log \int p(X, z) dz$$

$$= \log \int_{z_{11}} \int_{z_{12}} \dots \int_{z_{1K}} \dots \int_{z_{NK}} p_{\theta}(x_{11}, x_{12}, \dots, z_{11}, \dots, z_{NK}) dz_{11} dz_{12} \dots dz_{NK}$$

⇒ most of the times intractable

⇒ so rather, we introduce a lower bound to this expression

Evidence Lower Bound (ELBO)

- Jensen's inequality: for a concave function f :

$$f(E[x]) \geq E[f(x)]$$

proposed distr.

- derivation:

$$\log p_{\theta}(X) = \log \int p_{\theta}(X, z) \frac{q(z)}{q(z)} dz$$

$$= \log E_{q(z)} \left[\frac{p_{\theta}(X, z)}{q(z)} \right]$$

$$\geq E_{q(z)} \left[\log \frac{p_{\theta}(X, z)}{q(z)} \right]$$

$$= \text{ELBO}_{\theta}(q, z)$$

• Rewriting the ELBO in two ways:

$$1) \text{ELBO}_x(\theta, z) = E_q[\log p_\theta(x, z)]$$

$$- \underbrace{E_q[\log q(z)]}$$

$$= - \int q(z) \log q(z) \text{ entropy} \\ =: H(q(z))$$

$$= E_q[\log p_\theta(x, z)] + H(q(z))$$

$$2) E_q \left[\log \frac{p(x, z)}{q(z)} \right]$$

$$= E_q[\log(p(z|x)p(x)) - \log q(z)]$$

$$= E_q[\log p(z|x) - \log q(z)] + E_q[\log p(x)]$$

$$= - E_q[\log q(z) - \log p(z|x)] + \log p(x)$$

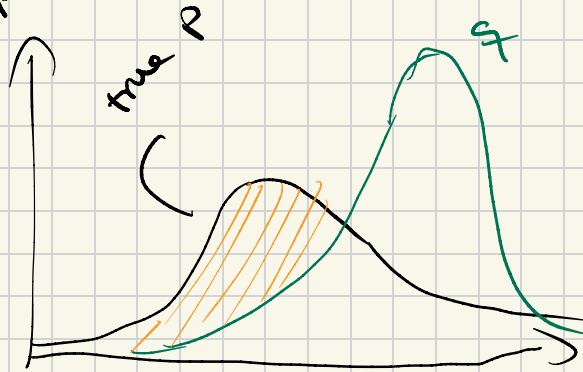
$$= - E_q \left[\log \frac{q(z)}{p(z|x)} \right] + \log p_\theta(x)$$

$$= -KL(q(z) \parallel p(z|x)) + \log p_\theta(x)$$

Remarks: • if $q(z) = p(z|x)$ then the bound becomes exact

• KL = Kullback-Leibler divergence and is a measure of how inefficient it is to assume q give a true distr. p ,

example:



underestimation
= "surprise"
most heavily
affects KL
div.

Expectation - Maximization (EM)

- EM is an iterative algorithm used to find maximum likelihood estimates when your data/model has hidden "latent" variables

- It alternates between:

1) Expectation step (E-step):

estimate hidden variables / posteriors

give a current parameter guess.

(e.g. guess which cluster data comes from)

2) Maximization step (M-step):

update model parameters assuming fixed

posterior (update cluster centers...)

- Write EM in terms of ELBO:

1) E-step:

$$\begin{aligned}
 q^*(z) &= \underset{q}{\operatorname{argmax}} \mathbb{E}_{q(z)} \log p(z|x, \overset{\text{fixed } \theta}{\theta^*}) \\
 &= \underset{q}{\operatorname{argmin}} \operatorname{KL}[q(z) \parallel p_\theta(z|x)]
 \end{aligned}$$

- if we can derive $p_\theta(z|x)$ analytically, this means we want to set $q(z) = p_\theta(z|x)$

2) M-step:

$$\begin{aligned}\theta^* &= \underset{\theta}{\operatorname{argmax}} \operatorname{ELBO}(\overset{\text{fix } \eta}{\underset{\sim}{\eta^*}}, \theta) \\ &= \underset{\theta}{\operatorname{argmax}} E_{\eta} [\log p_{\theta}(X, z)]\end{aligned}$$

Integrated EM algorithm:

- initialize param. estimates $\theta^{(0)}, z^{(0)}, \eta$
- while $\Delta \operatorname{ELBO} > \epsilon$
 - E-step
 - M-step

Continued Example: Parameter inference in GMMs

E-step:

Given $\theta^{(n)}$, we want to sample and to components k , the posterior:

$$p(z_{ik} = 1 \mid X, \theta^{(n)}) = \frac{p(x_i \mid \theta^{(n)}, z_{ik} = 1) \overset{\pi_k}{\underbrace{p(z_{ik} = 1)}}}{p(x_i \mid \theta^{(n)})}$$

$$= \frac{N(\mu_k, \Sigma_k) \pi_k}{\sum_{l=1}^K N(\mu_l, \Sigma_l) \pi_l}$$

M-step: we want $\max_z E_z [\log p_0(x, z)]$

$$\begin{aligned} & E_{z|x} [\log p(x, z)] \\ &= E_{z|x} [\log \prod_i p_0(x_i, z_i)] \\ &= E_{z|x} \left[\sum_i \log p(x_i | z_i) p(z_i) \right] \\ &= \sum_z p(z|x) \sum_i \log p(x_i | z_i) p(z_i) \\ &= \sum_i^N \sum_{k=1}^K \underbrace{p(z_{ik}=1 | x_i)}_{\substack{\text{known from the} \\ \text{E-step}}} \left[\underbrace{\log p_0(x_i | z_i)}_{\log N(\mu_k, \Sigma_k)} + \underbrace{\log p(z_i)}_{\log \pi_k} \right] \end{aligned}$$

$\Rightarrow$ here you can now optimize w.r.t. μ_k, Σ_k, π_k

Find EM algo for GMMs:

- initialize: $K, \mu_k^{(0)}, \Sigma_k^{(0)}, \pi_k^{(0)} \text{ } \theta^{(0)}$

- iterate:

E-step:

solve for $p(z_{ik} = 1 | x_i, \theta^{(n)}) := \gamma_{ik}$

M-step:

Updates for parameters:

$$\mu_k^{(n+1)} = \frac{1}{N_k} \sum_{i=1}^N \gamma_{ik} x_i$$

$$\Sigma_k^{(n+1)} = \frac{1}{N_k} \sum_{i=1}^N \gamma_{ik} (x_i - \mu_k^{(n+1)}) (x_i - \mu_k^{(n+1)})^T$$

$$\pi_k^{(n+1)} = \frac{N_k}{N} \quad N_k = \sum_{i=1}^N \gamma_{ik}$$

Remarks:

- You can use GMM for classifying new data points or for density estimation (= universal approximators of prob. densities)

- Determine optimal number K by plotting

ELBO against number of components and
select K at elbow.